

Loan Approval Prediction

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call

This kernel I want try to make a loan approval prediction based on [Loan Prediction](#) dataset. Previously, I was made the same work for my university couse task but with different dataset. You can see my previous work on [this link](#) (in Bahasa). This work will compare the result of loan approval prediction classiffication algorithm with different dataset from my previous work. In previous work I get the Gradient Boosting Classifier as the best classifier for loan aproval prediction. Beside that, I want to make some improvement in this kernel inspired by the work of [Baligh's](#).

✓ 1. Import Packages & Data

```
#Import packages
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn import svm

#Read CSV data
data = pd.read_csv("loan_prediction.csv")
```

```
#preview data
data.head()
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 614 entries, 0 to 613
Data columns (total 13 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Loan_ID         614 non-null    object
1   Gender          601 non-null    object
```

```

2   Married          611 non-null   object
3   Dependents       599 non-null   object
4   Education        614 non-null   object
5   Self_Employed    582 non-null   object
6   ApplicantIncome  614 non-null   int64
7   CoapplicantIncome 614 non-null   float64
8   LoanAmount       592 non-null   float64
9   Loan_Amount_Term 600 non-null   float64
10  Credit_History    564 non-null   float64
11  Property_Area     614 non-null   object
12  Loan_Status       614 non-null   object
dtypes: float64(4), int64(1), object(8)
memory usage: 67.2+ KB

```

✓ 2. Data Quality & Missing Value Assesment

```

#Preview data information
data.info()

```

```

<class 'pandas.core.frame.DataFrame'>
Index: 614 entries, 0 to 613
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Loan_ID               614 non-null   object
1   Gender                601 non-null   object
2   Married               611 non-null   object
3   Dependents            599 non-null   object
4   Education             614 non-null   object
5   Self_Employed         582 non-null   object
6   ApplicantIncome       614 non-null   int64
7   CoapplicantIncome     614 non-null   float64
8   LoanAmount            592 non-null   float64
9   Loan_Amount_Term      600 non-null   float64
10  Credit_History         564 non-null   float64
11  Property_Area          614 non-null   object
12  Loan_Status           614 non-null   object
dtypes: float64(4), int64(1), object(8)
memory usage: 67.2+ KB

```

```

#Check missing values
data.isnull().sum()

```

	0
Loan_ID	0
Gender	13
Married	3
Dependents	15
Education	0
Self_Employed	32
ApplicantIncome	0
CoapplicantIncome	0
LoanAmount	22
Loan_Amount_Term	14
Credit_History	50
Property_Area	0
Loan_Status	0

dtype: int64

✓ Gender - Missing Values

```
# percent of missing "Gender"
print('Percent of missing "Gender" records is %.2f%%' %((data['Gender'].isnu
```

Percent of missing "Gender" records is 2.12%

```
print("Number of people who take a loan group by gender :")
print(data['Gender'].value_counts())
sns.countplot(x='Gender', data=data, palette = 'Set2')
```

Number of people who take a loan group by gender :

Gender

Male 489

Female 112

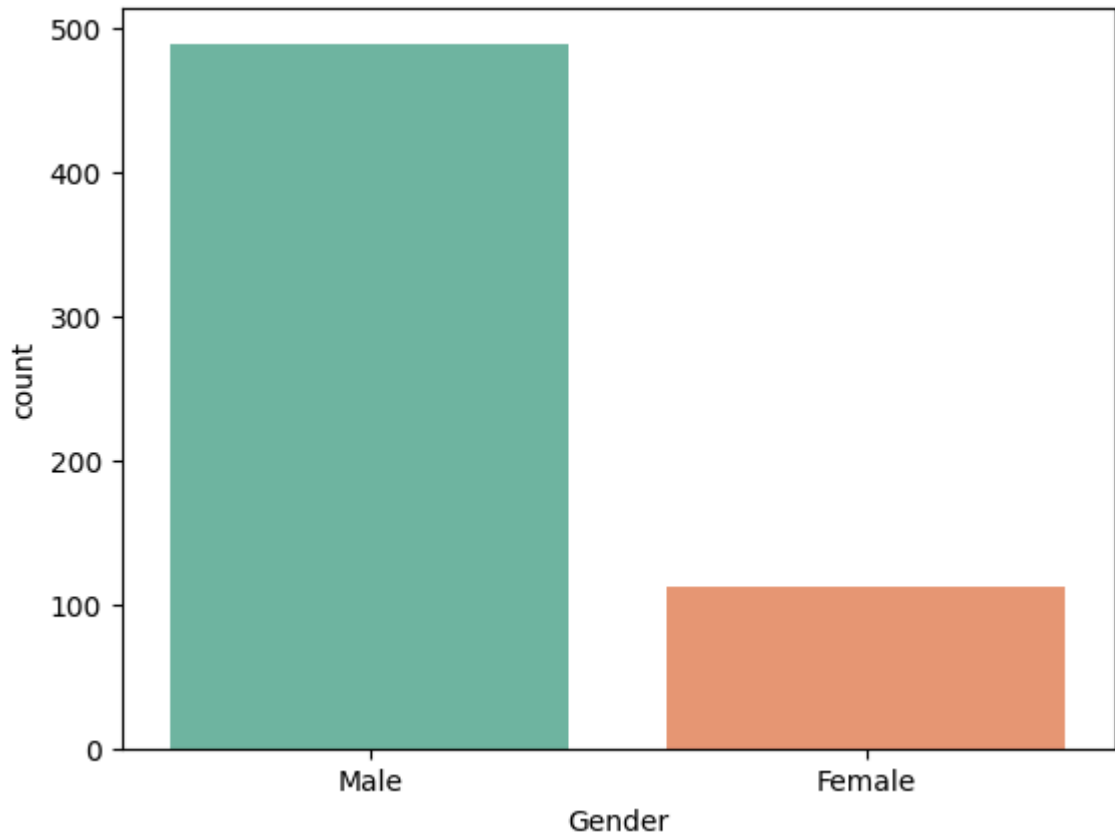
Name: count, dtype: int64

/tmp/ipykernel_3443/1357974753.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed i

```
sns.countplot(x='Gender', data=data, palette = 'Set2')
```

```
<Axes: xlabel='Gender', ylabel='count'>
```



✓ Married - Missing Values

```
# percent of missing "Married"
```

```
print('Percent of missing "Married" records is %.2f%%' %((data['Married'].is
```

```
Percent of missing "Married" records is 0.49%
```

```
print("Number of people who take a loan group by marital status :")
```

```
print(data['Married'].value_counts())
```

```
sns.countplot(x='Married', data=data, palette = 'Set2')
```

Number of people who take a loan group by marital status :

Married

Yes 398

No 213

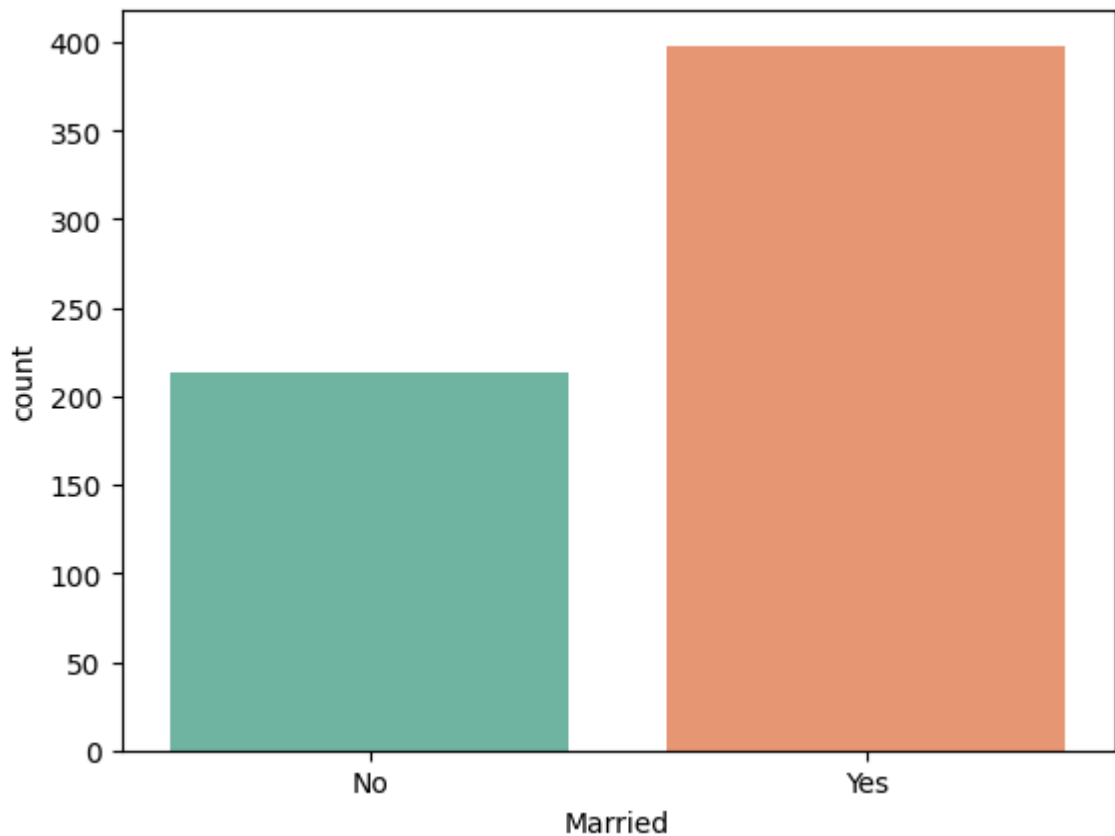
Name: count, dtype: int64

/tmp/ipykernel_3443/1513261366.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed i

```
sns.countplot(x='Married', data=data, palette = 'Set2')
```

```
<Axes: xlabel='Married', ylabel='count'>
```



✓ Dependents- Missing Values

```
# percent of missing "Dependents"
```

```
print('Percent of missing "Dependents" records is %.2f%%' %((data['Dependent
```

```
Percent of missing "Dependents" records is 2.44%
```

```
print("Number of people who take a loan group by dependents :")
```

```
print(data['Dependents'].value_counts())
```

```
sns.countplot(x='Dependents', data=data, palette = 'Set2')
```

Number of people who take a loan group by dependents :

Dependents

0 345

1 102

2 101

3+ 51

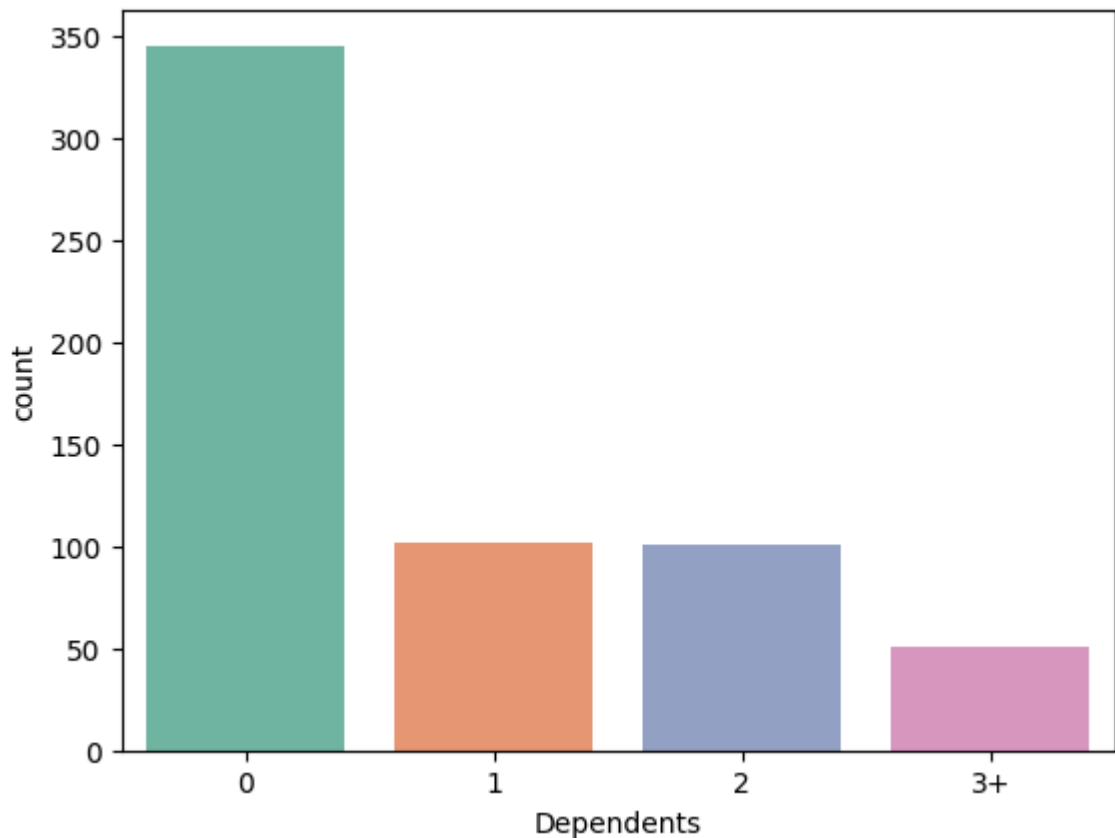
Name: count, dtype: int64

/tmp/ipykernel_3443/748461053.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed i

```
sns.countplot(x='Dependents', data=data, palette = 'Set2')
```

<Axes: xlabel='Dependents', ylabel='count'>



✓ Self Employed - Missing Values

```
# percent of missing "Self_Employed"
```

```
print('Percent of missing "Self_Employed" records is %.2f%%' %((data['Self_E
```

```
Percent of missing "Self_Employed" records is 5.21%
```

```
print("Number of people who take a loan group by self employed :")
```

```
print(data['Self_Employed'].value_counts())
```

```
sns.countplot(x='Self_Employed', data=data, palette = 'Set2')
```

Number of people who take a loan group by self employed :

Self_Employed

No 500

Yes 82

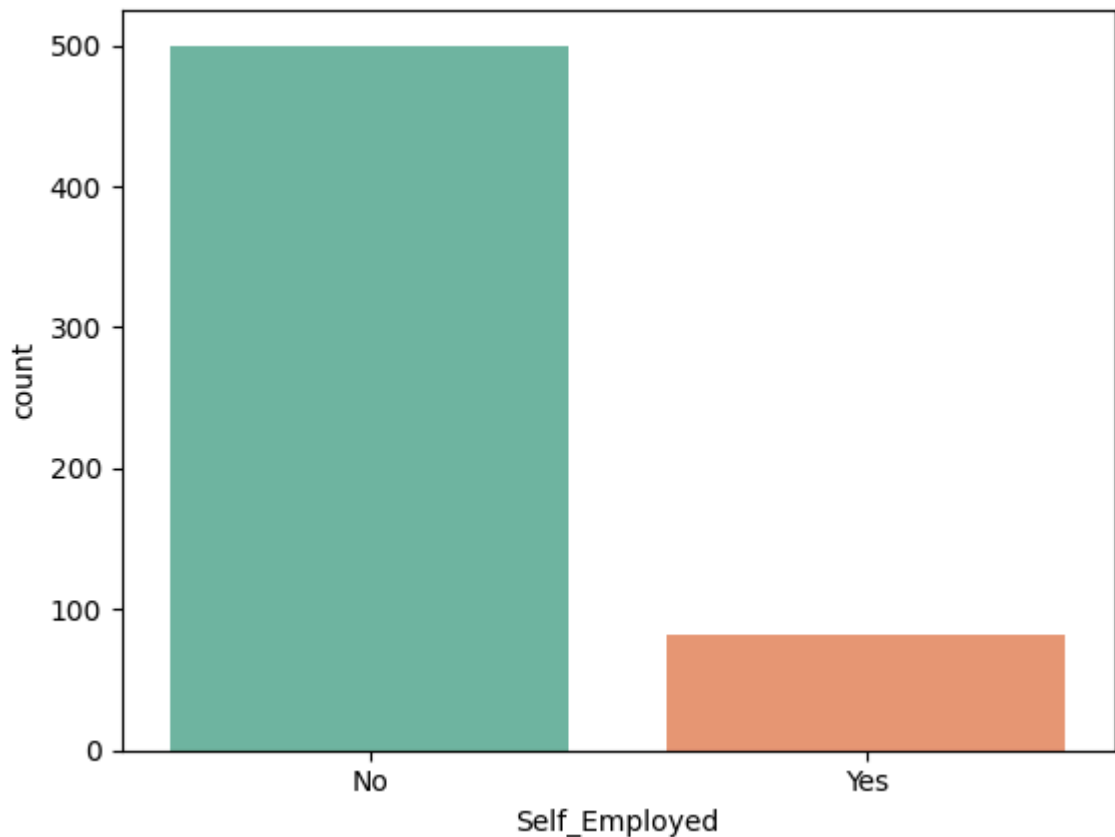
Name: count, dtype: int64

/tmp/ipykernel_3443/2034259608.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed i

```
sns.countplot(x='Self_Employed', data=data, palette = 'Set2')
```

```
<Axes: xlabel='Self_Employed', ylabel='count'>
```



✓ Loan Amount - Missing Values

```
# percent of missing "LoanAmount"
```

```
print('Percent of missing "LoanAmount" records is %.2f%%' %((data['LoanAmount'].isnull().sum() / data['LoanAmount'].count() * 100)))
```

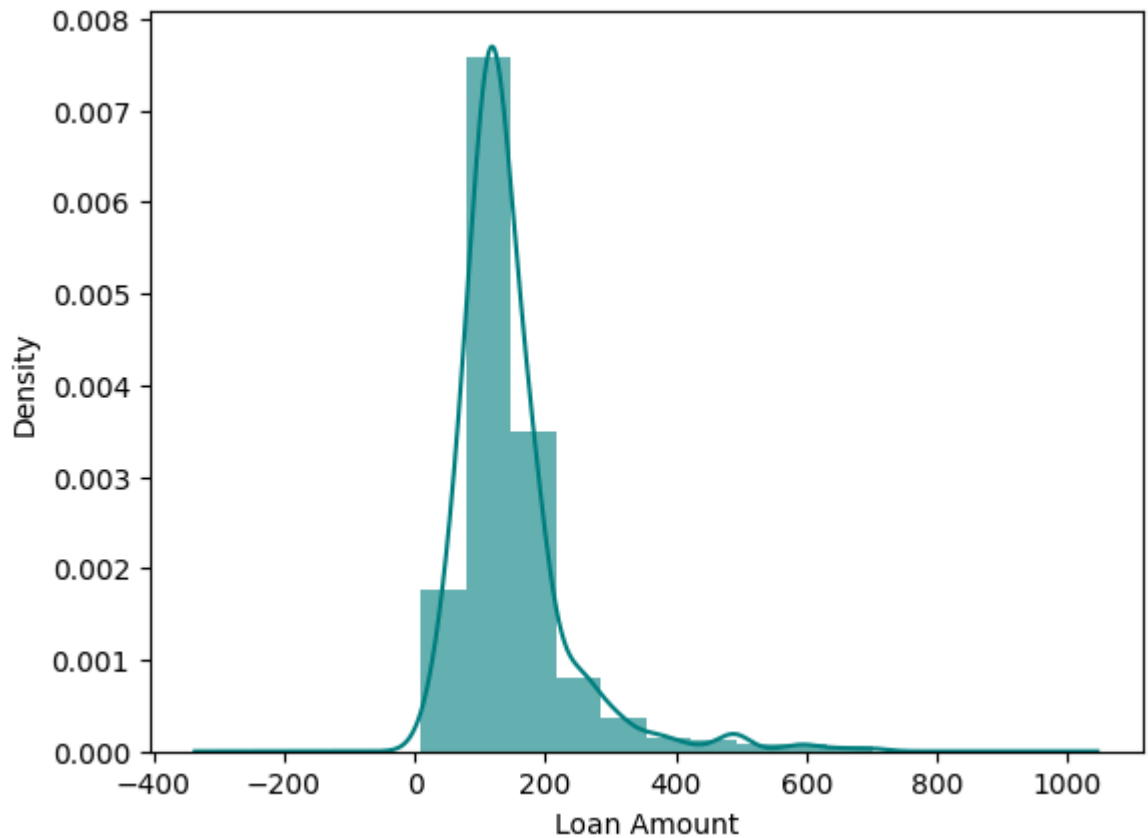
```
Percent of missing "LoanAmount" records is 3.58%
```

```
ax = data["LoanAmount"].hist(density=True, stacked=True, color='teal', alpha=0.5)
```

```
data["LoanAmount"].plot(kind='density', color='teal')
```

```
ax.set(xlabel='Loan Amount')
```

```
plt.show()
```



✓ Loan Amount Term - Missing Values

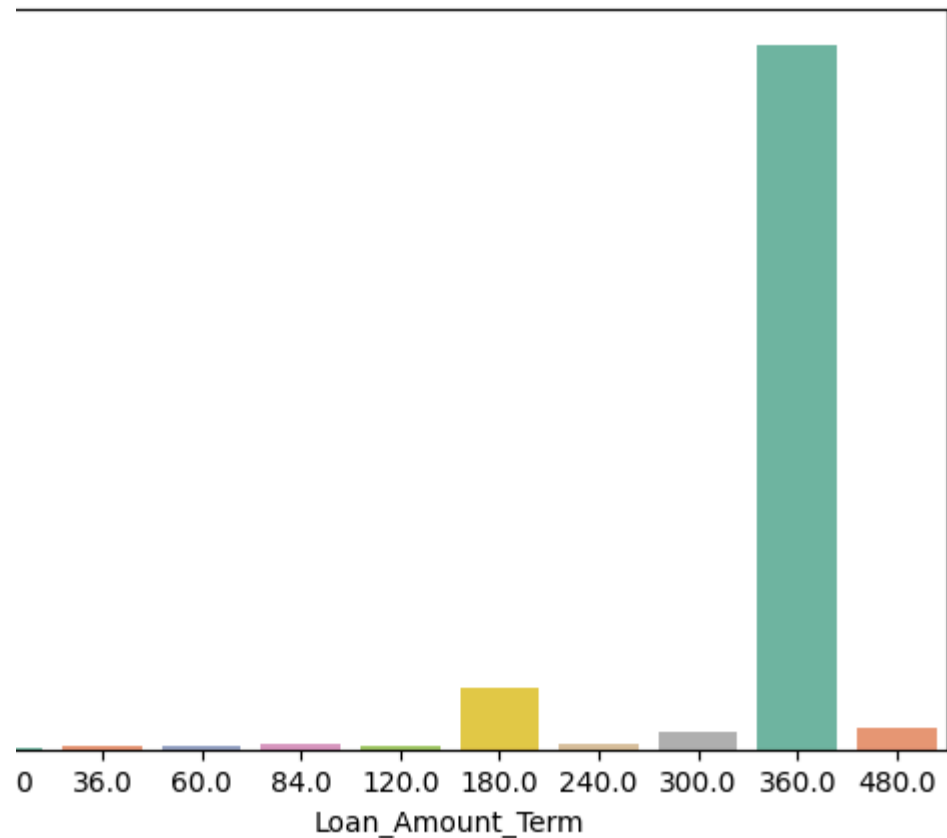
```
# percent of missing "Loan_Amount_Term"
print('Percent of missing "Loan_Amount_Term" records is %.2f%%' %((data['Loa
```

```
Percent of missing "Loan_Amount_Term" records is 2.28%
```

```
print("Number of people who take a loan group by loan amount term :")
print(data['Loan_Amount_Term'].value_counts())
sns.countplot(x='Loan_Amount_Term', data=data, palette = 'Set2')
```


people who take a loan group by loan amount term :

```
dtype: int64
1_3443/1446952119.py:3: FutureWarning:
palette` without assigning `hue` is deprecated and will be removed in
plot(x='Loan_Amount_Term', data=data, palette = 'Set2')
l='Loan_Amount_Term', ylabel='count'>
```



✓ Credit History - Missing Values

```
# percent of missing "Credit_History"
print('Percent of missing "Credit_History" records is %.2f%%' % ((data['Credi
```

Percent of missing "Credit_History" records is 8.14%

```
print("Number of people who take a loan group by credit history :")  
print(data['Credit_History'].value_counts())  
sns.countplot(x='Credit_History', data=data, palette = 'Set2')
```

Number of people who take a loan group by credit history :

Credit_History

1.0 475

0.0 89

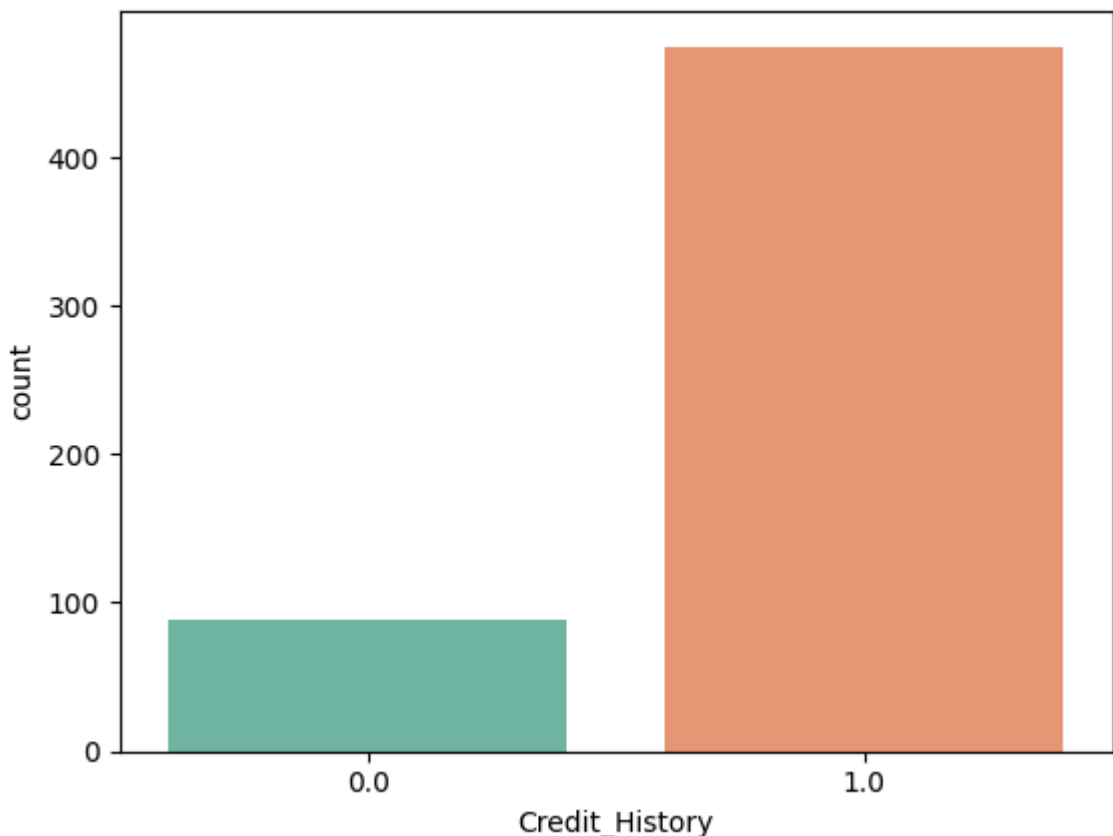
Name: count, dtype: int64

/tmp/ipykernel_3443/592015175.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed i

```
sns.countplot(x='Credit_History', data=data, palette = 'Set2')
```

<Axes: xlabel='Credit_History', ylabel='count'>



✓ 3. Final Adjustments to Data

Based on my assessment of the missing values in the dataset, I'll make the following changes to the data:

- If "Gender" is missing for a given row, I'll impute with Male (most common answer).
- If "Married" is missing for a given row, I'll impute with yes (most common answer).
- If "Dependents" is missing for a given row, I'll impute with 0 (most common answer).

- If "Self_Employed" is missing for a given row, I'll impute with no (most common answer).
- If "LoanAmount" is missing for a given row, I'll impute with mean of data.
- If "Loan_Amount_Term" is missing for a given row, I'll impute with 360 (most common answer).
- If "Credit_History" is missing for a given row, I'll impute with 1.0 (most common answer).

```
train_data = data.copy()
train_data['Gender'].fillna(train_data['Gender'].value_counts().idxmax(), inplace=True)
train_data['Married'].fillna(train_data['Married'].value_counts().idxmax(), inplace=True)
train_data['Dependents'].fillna(train_data['Dependents'].value_counts().idxmax(), inplace=True)
train_data['Self_Employed'].fillna(train_data['Self_Employed'].value_counts().idxmax(), inplace=True)
train_data['LoanAmount'].fillna(train_data['LoanAmount'].mean(skipna=True), inplace=True)
train_data['Loan_Amount_Term'].fillna(train_data['Loan_Amount_Term'].value_counts().idxmax(), inplace=True)
train_data['Credit_History'].fillna(train_data['Credit_History'].value_counts().idxmax(), inplace=True)
```

3443/1279629814.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series and its inplace parameter will be ignored in pandas 3.0. This inplace method will never work because the operation is not supported. In pandas 3.0, the operation will raise an error. In the meantime, you can avoid this warning by using 'df[col].method(value, inplace=True)', try using 'df.method({col: value, inplace=True})'.

3443/1279629814.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series and its inplace parameter will be ignored in pandas 3.0. This inplace method will never work because the operation is not supported. In pandas 3.0, the operation will raise an error. In the meantime, you can avoid this warning by using 'df[col].method(value, inplace=True)', try using 'df.method({col: value, inplace=True})'.

3443/1279629814.py:4: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series and its inplace parameter will be ignored in pandas 3.0. This inplace method will never work because the operation is not supported. In pandas 3.0, the operation will raise an error. In the meantime, you can avoid this warning by using 'df[col].method(value, inplace=True)', try using 'df.method({col: value, inplace=True})'.

3443/1279629814.py:5: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series and its inplace parameter will be ignored in pandas 3.0. This inplace method will never work because the operation is not supported. In pandas 3.0, the operation will raise an error. In the meantime, you can avoid this warning by using 'df[col].method(value, inplace=True)', try using 'df.method({col: value, inplace=True})'.

3443/1279629814.py:6: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series and its inplace parameter will be ignored in pandas 3.0. This inplace method will never work because the operation is not supported. In pandas 3.0, the operation will raise an error. In the meantime, you can avoid this warning by using 'df[col].method(value, inplace=True)', try using 'df.method({col: value, inplace=True})'.

3443/1279629814.py:7: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series and its inplace parameter will be ignored in pandas 3.0. This inplace method will never work because the operation is not supported. In pandas 3.0, the operation will raise an error. In the meantime, you can avoid this warning by using 'df[col].method(value, inplace=True)', try using 'df.method({col: value, inplace=True})'.

```

    en doing 'df[col].method(value, inplace=True)', try using 'df.method({col: val

    an_Amount_Term'].fillna(train_data['Loan_Amount_Term'].value_counts().idxmax(
3443/1279629814.py:8: FutureWarning: A value is trying to be set on a copy of
ll change in pandas 3.0. This inplace method will never work because the inter

    en doing 'df[col].method(value, inplace=True)', try using 'df.method({col: val

    redit_History'].fillna(train_data['Credit_History'].value_counts().idxmax(), i

```

```

#Check missing values
train_data.isnull().sum()
train_data

```

Loan_ID	Gender	Married	Dependents	Education	Self_Employed	Applicant
'001002	Male	No	0	Graduate	No	
'001003	Male	Yes	1	Graduate	No	
'001005	Male	Yes	0	Graduate	Yes	
'001006	Male	Yes	0	Not Graduate	No	
'001008	Male	No	0	Graduate	No	
...	...	...	...	...	...	...
'002978	Female	No	0	Graduate	No	
'002979	Male	Yes	3+	Graduate	No	
'002983	Male	Yes	1	Graduate	No	
'002984	Male	Yes	2	Graduate	No	
'002990	Female	No	0	Graduate	Yes	

× 13 columns

Next steps: [Generate code with train_data](#)

[New interactive sheet](#)

```

#Convert some object data type to int64
gender_stat = {"Female": 0, "Male": 1}
yes_no_stat = {'No' : 0, 'Yes' : 1}
dependents_stat = {'0':0, '1':1, '2':2, '3+':3}
education_stat = {'Not Graduate' : 0, 'Graduate' : 1}
property_stat = {'Semiurban' : 0, 'Urban' : 1, 'Rural' : 2}

train_data['Gender'] = train_data['Gender'].replace(gender_stat)
train_data['Married'] = train_data['Married'].replace(yes_no_stat)
train_data['Dependents'] = train_data['Dependents'].replace(dependents_stat)
train_data['Education'] = train_data['Education'].replace(education_stat)

```

```
train_data['Self_Employed'] = train_data['Self_Employed'].replace(yes_no_sta
train_data['Property_Area'] = train_data['Property_Area'].replace(property_s
```

```
/tmp/ipykernel_3443/3463757473.py:8: FutureWarning: Downcasting behavior in `
  train_data['Gender'] = train_data['Gender'].replace(gender_stat)
/tmp/ipykernel_3443/3463757473.py:9: FutureWarning: Downcasting behavior in `
  train_data['Married'] = train_data['Married'].replace(yes_no_stat)
/tmp/ipykernel_3443/3463757473.py:10: FutureWarning: Downcasting behavior in
  train_data['Dependents'] = train_data['Dependents'].replace(dependents_stat)
/tmp/ipykernel_3443/3463757473.py:11: FutureWarning: Downcasting behavior in
  train_data['Education'] = train_data['Education'].replace(education_stat)
/tmp/ipykernel_3443/3463757473.py:12: FutureWarning: Downcasting behavior in
  train_data['Self_Employed'] = train_data['Self_Employed'].replace(yes_no_st
/tmp/ipykernel_3443/3463757473.py:13: FutureWarning: Downcasting behavior in
  train_data['Property_Area'] = train_data['Property_Area'].replace(property_
```

```
#Preview data information
data.info()
data.isnull().sum()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 614 entries, 0 to 613
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Loan_ID                614 non-null   object
1   Gender                 601 non-null   object
2   Married                611 non-null   object
3   Dependents             599 non-null   object
4   Education              614 non-null   object
5   Self_Employed          582 non-null   object
6   ApplicantIncome        614 non-null   int64
7   CoapplicantIncome      614 non-null   float64
8   LoanAmount             592 non-null   float64
9   Loan_Amount_Term       600 non-null   float64
10  Credit_History         564 non-null   float64
11  Property_Area          614 non-null   object
12  Loan_Status            614 non-null   object
dtypes: float64(4), int64(1), object(8)
memory usage: 67.2+ KB
```

	0
Loan_ID	0
Gender	13
Married	3
Dependents	15
Education	0
Self_Employed	32
ApplicantIncome	0
CoapplicantIncome	0
LoanAmount	22
Loan_Amount_Term	14
Credit_History	50
Property_Area	0
Loan_Status	0

dtype: int64

✓ 4. Making Prediction

```
#Separate feature and target
x = train_data.iloc[:,1:12]
y = train_data.iloc[:,12]

#make variabel for save the result and to show it
classifier = ('Gradient Boosting','Random Forest','Decision Tree','K-Neares
```

```
y_pos = np.arange(len(classifier))  
score = []
```

```
clf = GradientBoostingClassifier()  
scores = cross_val_score(clf, x, y, cv=5)  
score.append(scores.mean())  
print('The accuration of classification is %.2f%%' %(scores.mean()*100))
```

The accuration of classification is 78.01%

```
clf = RandomForestClassifier(n_estimators=10)  
scores = cross_val_score(clf, x, y, cv=5)  
score.append(scores.mean())  
print('The accuration of classification is %.2f%%' %(scores.mean()*100))
```

The accuration of classification is 76.22%

```
clf = DecisionTreeClassifier()  
scores = cross_val_score(clf, x, y, cv=5)  
score.append(scores.mean())  
print('The accuration of classification is %.2f%%' %(scores.mean()*100))
```

The accuration of classification is 70.85%

```
clf = KNeighborsClassifier()  
scores = cross_val_score(clf, x, y, cv=5)  
score.append(scores.mean())  
print('The accuration of classification is %.2f%%' %(scores.mean()*100))
```

The accuration of classification is 61.40%

```
clf = svm.LinearSVC(max_iter=5000)
```